

ArtBridge API 명세서 (코드 기준 v0.8)

라우트 파일 43개(`src/app/api/**/route.ts`) · HTTP 메서드 핸들러 56개(파일당 GET/POST 등 복수 메서드 포함). 기준: `bigfive2026/cadet-project main`. 인증은 `Auth.js(JWT)` 세션.

프로그램(Program) 도메인 API

문서화한 라우트 파일은 6개, 총 엔드포인트(HTTP 메서드 export)는 11개다. 모든 행은 `src/app/api/programs/**/route.ts` 와 그 서비스(`src/lib/*`)·검증 스키마(`src/lib/validation/*`)의 실제 코드를 근거로 한다.

Method	Endpoint	기능	인증·역할	
GET	<code>/api/programs</code>	공개 프로그램 목록 (<code>deletedAt IS NULL</code> + 공개 상태)	공개	쿼리: <code>category?</code>
POST	<code>/api/programs</code>	프로그램 생성 (<code>status</code> 는 일정 기반 자동 산정, 기본 <code>RECRUITING</code>)	로그인필요 → CREATOR(본인 <code>creatorProfile</code> 필요)	<code>body(zod pr</code> <code>title(1~20C</code> <code>(≤5000), pri</code> <code>category?(≤</code> <code>startDate?(/</code> <code>(date), maxPa</code>
POST	<code>/api/programs/ai-suggest</code>	AI 가격·혜택·주차 구성 추천 (OpenAI 또는 Mock 폴백)	로그인필요 → CREATOR	<code>body(zod): d</code> <code>duration?, c</code> <code>targetAudie</code>
GET	<code>/api/programs/{id}</code>	공개 상세 조회 (<code>soft-delete</code> 제외)	공개	<code>path: id</code>
PATCH	<code>/api/programs/{id}</code>	프로그램 수정 (일정 변경 시 <code>status</code> 재계산, <code>CLOSED</code> 전이 시 알림)	로그인필요 → CREATOR + 본인 소유	<code>body(zod pr</code> <code>필드): title?</code> <code>priceKrw?, c</code> <code>startDate?(/</code> <code>(nullable dat</code> <code>(nullable)</code>
DELETE	<code>/api/programs/{id}</code>	프로그램 <code>soft delete</code> (<code>deletedAt</code> 설정, 물리삭제 금지)	로그인필요 → CREATOR + 본인 소유	<code>path: id</code>
POST	<code>/api/programs/{id}/complete</code>	완료 처리(크리에이터 일괄 납품 요청, SPEC-013 예	로그인필요 → CREATOR + 본	<code>path: id (bo</code>

		스크로로 재정의)	인 소유	
POST	/api/programs/{id}/applications	프로그램 참여 신청 (무료=즉시 ACCEPTED, 유료=PENDING_PAYMENT/결제)	로그인필요 (FAN/CREATOR 모두, 자기 프로그램은 CREATOR 역할로 신청 불가)	path: id; body: message?(≤1000)
GET	/api/programs/{id}/applications	크리에이터 본인 프로그램의 신청 목록 (최신순)	로그인필요 → CREATOR	path: id
GET	/api/programs/{id}/reviews	프로그램 리뷰 목록 + 평균 평점(크리에이터가 받은 평점만)	공개	path: id
POST	/api/programs/{id}/reviews	리뷰 작성 (양방향: 팬→크리에이터 / 크리에이터→팬)	로그인필요 (완료 승인된 참여자/소유 크리에이터)	path: id; body: rating(int 1-5, 공백→null), text(≤1000), reviewer

비고: 표의 {id} 는 Next App Router의 [id] 동적 세그먼트다. /api/programs/{id}/complete 의 인증·역할은 라우트에서 비로그인(401)만 직접 차단하고, 나머지 CREATOR·소유권(403/404/400)은 completeProgram 서비스 결과 status로 위임된다(applications·reviews POST도 동일 패턴). AI 추천은 서비스 예외 시에도 키 노출 방지를 위해 200으로 응답한다.

프로그램 신청·에스크로 (Application)

전수 분석한 라우트 파일(전부 실제 코드 근거):

- src/app/api/programs/[id]/applications/route.ts (POST, GET)
- src/app/api/applications/[id]/route.ts (PATCH)
- src/app/api/applications/[id]/contract/route.ts (POST)
- src/app/api/applications/[id]/request-delivery/route.ts (POST)

- `src/app/api/applications/{id}/approve-completion/route.ts` (POST)

Method	Endpoint	기능
(참조)	<code>/api/programs/{id}/applications</code>	참여 신청(POST)·신청목록 조회(GET) — 경로상 프로그램 하위라 위 프로그램 도메인 표에 상세 기재
PATCH	<code>/api/applications/{id}</code>	신청 취소(cancel, 팬 본인) 또는 확정 참여자 제외(remove, 크리에이터). PAID면 환불·정산 ON_HOLD 처리
POST	<code>/api/applications/{id}/contract</code>	ACCEPTED 신청에 대한 계약 생성/조회(먹등, terms 스냅샷)
POST	<code>/api/applications/{id}/request-delivery</code>	납품 요청(크리에이터 → PAID 참여, IN_PROGRESS 한정, 먹등). 팬에게 DELIVERY_REQUESTED 알림
POST	<code>/api/applications/{id}/approve-completion</code>	에스크로 완료 승인(팬=지불자). Payment/Settlement RELEASED + completionApprovedAt + 전원 승인 시 Program COMPLETED, COMPLETION_APPROVED·MUTUAL_REVIEW_REQUESTED 알림

비고:

- 모든 라우트는 `getCurrentUser()` (`@/lib/auth`)로 세션 기반 인증. 미로그인 시 일괄 401.
- 역할 판정은 라우트(GET의 CREATOR 체크)와 서비스 계층(`ensureCreator` / `resolveAccess` /지불자 일치)에서 이중으로 수행됨.
- "Mock폴백"은 결제 provider에 국한된 동작이며, 신청·에스크로 도메인 로직 자체는 실 Prisma 트랜잭션으로 모두 구현완료. 부분구현/스텝/순수 Mock인 엔드포인트는 없음.

계약·서명·결제 (Contract)

전수 대상: `/api/contracts/{id}` 하위 6개 route.ts. 각 라우트는 단일 HTTP 메서드만 export(PATCH 5개 + POST 1개). 모든 인증은 `getCurrentUser()` (세션) 기반이며, 서비스 컨텍스트로 `{ userId, role, creatorProfileId }` 를 전달한다. 역할 게이트는 라우트가 아니라 서비스 계층(`resolveAccess` 또는 `userId` 직접 비교)에서 수행한다. 모든 서비스는 `src/lib/contracts.ts` 에 실제 Prisma 트랜잭션으로 구현됨 → 라우트 단위 구현상태는 모두 [구현완료]. 단, 결

제(payment)는 결제 Provider가 환경변수 미설정 시 `MockPaymentProvider` 로 폴백되어 외부 네트워크 없이 즉시 PAID 처리하는 [Mock폴백] 경로를 포함한다.

Method	Endpoint	기능	인증·역
PATCH	<code>/api/contracts/{id}/amount</code>	합의 금액 제시(크리에이터가 양의 정수 금액 제시, <code>agreedAmount</code> 갱신 + <code>terms.amountProposedAt</code> 기록 + 팬에게 <code>CONTRACT_AMOUNT_PROPOSED</code> 알림)	로그인필요 / CREATE(유자만, <code>resolveAccess===</code>)
PATCH	<code>/api/contracts/{id}/amount/agree</code>	합의 금액 동의(팬 본인이 제시 금액 동의, <code>terms.amountAgreedAt</code> 기록. 미제시·거부 상태면 400, 이미 동의 시 먹등 반환)	로그인필요 / FAN(팬 <code>userId===applica</code>)
PATCH	<code>/api/contracts/{id}/amount/reject</code>	합의 금액 거부·결렬(팬 본인. 단일 트랜잭션으로 신청 <code>REJECTED</code> + <code>terms.amountRejectedAt</code> + 프로그램 <code>CONTRACTING→RECRUITING</code> 복귀 + 크리에이터에게 <code>CONTRACT_AMOUNT_REJECTED</code> 알림)	로그인필요 / FAN(팬)
PATCH	<code>/api/contracts/{id}/sign</code>	팬 전자 서명(팬 본인. 금액 제시됐으나 미합의 시 선행 동의 요구. <code>fanSignedAt</code> 설정(먹등). 양측 서명 완료 시 <code>CONTRACT_SIGNED</code> 알림)	로그인필요 / FAN(팬 <code>resolveAccess===</code>)
PATCH	<code>/api/contracts/{id}/sign/creator</code>	크리에이터 전자 서명(프로그램 소유 크리에이터. <code>creatorSignedAt</code> 설정(먹등). 양측 서명 완료 시 <code>CONTRACT_SIGNED</code> 알림)	로그인필요 / CREATE(유자만, <code>resolveAccess===</code>)
POST	<code>/api/contracts/{id}/payment</code>	결제 시작(팬 본인. 양측 서명 완료 가드 후 <code>agreedAmount</code> 기준 결제. Mock 폴백이면 즉시 PAID 트랜잭션 (<code>Payment+Settlement+Program IN_PROGRESS+알림</code>), <code>sandbox</code> 면 <code>PENDING Payment</code> 생성 후 결제창 메타 반환)	로그인필요 / FAN(팬 <code>userId===applica</code>)

비고:

- 라우트 처리 순서는 공통적으로 비로그인 401 → JSON 파싱 실패 400 → zod 검증 실패 400 → 서비스 결과 (403/404/409/500) → 200.
- `verifyPayment` (결제 검증·확정) 서비스 함수는 `src/lib/contracts.ts` 에 구현돼 있으나, `src/app/api/contracts/` 하위에는 이를 노출하는 콜백/검증 `route.ts`가 존재하지 않음(이 디렉터리 전수 기준 미노출). GET 메서드를 export하는 라우트는 이 디렉터리에 없음(계약 생성·조회 `getOrCreateContract` 도 `contracts` 디렉터리 밖에서 호출됨).

- paymentSchema 의 provider 필드는 검증만 통과시키고 실제 분기에 미사용 → 서버 권위 검증(클라이언트 provider 불신뢰) 의도.

결제 콜백(Payment)

대상 디렉터리 /src/app/api/payments 전수 조사 결과, route.ts는 callback/route.ts 단 1개이며 POST / GET 두 메서드를 export한다. 두 메서드 모두 동일하게 verifyPayment() 서비스(src/lib/contracts.ts)로 위임한다.

Method	Endpoint	기능	인증.역할	요청.응답
POST	/api/payments/callback	PG 결제 검증·확정. JSON 본문 {merchantUid, providerTxId} 수신 → PG 단건 조회로 금액·주문번호·상태 대조 후 성공 시에만 PAID 확정(정산 PENDING 생성 + 프로그램 IN_PROGRESS + 알림 생성). 먹 등 처리(PAID/RELEASED 재요청 시 200).	로그인필요 (getCurrentUser, 없으면 401). 추가로 서비스단에서 해당 계약의 팬 본인만 허용 (ctx.userId !== payment.fanUserId → 403). 별도 role 분기는 없음.	zod payment(merchantId: string(), providerTxId: string())는 request
GET	/api/payments/callback	PG 리다이렉트 쿼리 파라미터 형태 수용. ?merchantUid=&providerTxId= (또는 imp_uid 별칭)을 읽어 동일하게 검증·확정. POST와 완전히 동일한 verifyPayment 흐름.	로그인필요 (getCurrentUser, 없으면 401). 서비스단 팬 본인 검증(403) 동일.	쿼리스트링 providerTxId, imp_uid payment()로 동일 검

근거 파일:

- 라우트: `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/payments/callback/route.ts`
- 검증 스키마: `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/validation/contract.ts` (`paymentCallbackSchema`)
- 서비스: `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/contracts.ts` (`verifyPayment` , `L612-702`)
- Provider: `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/payment/provider.ts` (`resolvePaymentProvider` , `MockPaymentProvider` , `SandboxPaymentProvider.verifyPayment`)

작품 주문·배송 (ArtworkOrder)

대상 디렉터리 `/src/app/api/artwork-orders` 하위 라우트 전수 분석. 모든 라우트는 `getCurrentUser()` 로 로그인 검증 후 `@/lib/artwork-fulfillment` 의 서비스 함수에 위임하며, 인가(역할·소유권·상태 전이) 검증은 서비스 계층에서 수행한다. 서비스 5종 모두 실제 `Prisma $transaction` + 알림 생성으로 동작하고 `Mock` 폴백 분기가 없어 전부 **구현완료**.

Method	Endpoint	기능	인증·역할	
POST	<code>/api/artwork-orders/{id}/shipment</code>	작품 주문 발송 처리 (운송장 등록, 주문 SHIPPED, 구매자 알림)	로그인필요 → 서비스에서 CREATOR + 작품 소유자만 허용	<code>shipArtworkOrderScheme</code>
POST	<code>/api/artwork-orders/{id}/received</code>	구매자 수령 확정(주문 RECEIVED, 배송 deliveredAt, 정산 AVAILABLE 전환, 알림)	로그인필요 → 서비스에서 주문의 fanUserId(구매자) 본인만 허용	본문 없음 (params의 id만 사용)
POST	<code>/api/artwork-orders/{id}/issues</code>	구매자 배송/상품 이슈 신고(이슈 생성, 주문 ISSUE_OPENED, 정산 ON_HOLD, 작가 알림)	로그인필요 → 서비스에서 주문의 fanUserId(구매자) 본인만 허용	<code>reportArtworkIssueScheme</code> NOT_DELIVERED/DAMAGED message(1~2000), image
POST	<code>/api/artwork-orders/{id}/issues/resolve</code>	작가의 이슈 해결 처리 (OPEN·REVIEWING 이슈 RESOLVED, 주문 RECEIVED, 정산 AVAILABLE 전환, 구매자 알림)	로그인필요 → 서비스에서 CREATOR + 작품 소유자만 허용	<code>resolveArtworkIssueScheme</code>

POST	/api/artwork-orders/{id}/refund	작가의 환불 처리(주문 REFUNDED, 결제 REFUNDED, 정산 ADJUSTED·차감조정, 재고 복원, 구매자 알림)	로그인필요 → 서비스에서 CREATOR + 작품 소유자만 허용	refundArtworkOrderSch
------	---------------------------------	--	------------------------------------	-----------------------

근거 메모

- 라우트 파일 5개 모두 HTTP 메서드 export는 POST 단일 (GET/PATCH/PUT/DELETE 없음). 누락 메서드 없음.
- 인증: 라우트는 `getCurrentUser()` null 검사로 401만 처리하고, 역할/소유권 검증(403)·상태 검증(400)·존재 검증(404)·중복/충돌(409)은 전부 `src/lib/artwork-fulfillment.ts` 서비스 내부에서 반환.
- 검증 스키마 출처: `src/lib/validation/artwork-order.ts` (received는 스키마 없이 body 미파싱).
- `received · issues` (신고)는 구매자(`fanUserId`) 권한, `shipment · refund · issues/resolve`는 CREATOR+소유자 권한.
- 구현상태 판정: 5개 서비스 함수 모두 `prisma.$transaction` 으로 실제 DB 갱신·알림 생성 수행, Mock/스텝 분기 없음 → 전부 구현완료.

크리에이터 (작품·정산·업로드)

전체 6개 라우트 파일, 8개 엔드포인트(HTTP 메서드 기준)를 전수 문서화했다. 모든 라우트는 `getCurrentUser()` (실제 세션 → DB 조회, `include: { creatorProfile: true }`)로 인증하며, `user.role === "CREATOR"` 및 `user.creatorProfile` 존재를 공통 가드로 검사한다. 모든 데이터 접근은 실제 Prisma 쿼리 또는 실제 파일시스템 쓰기이며, 어떤 라우트에도 Mock 폴백이 존재하지 않는다.

Method	Endpoint	기능	인증·역할	
GET	/api/creator/settlements	본인(크리에이터)의 정산 내역 조회. 멤버십/포스트/계약/선착순프로그램/작품 결제 정산을 Settlement 기준으로 최신순 통합 조회	CREATOR (로그인 + role=CREATOR + creatorProfile 필수)	없음 (쿼리/바디 유효성 검사 없음)
POST	/api/creator/uploads	크리에이터 에셋 이미지 업로드. <code>public/uploads/creator-assets/</code> 에 파일 저장 후 공개 URL 반환	CREATOR	multipart/form-data 허용 타입 jpeg/png
GET	/api/creator/works	본인 활동/이력(CreatorWork) 목록 조회 (startedAt desc, createdAt desc)	CREATOR	없음
POST	/api/creator/works	활동/이력(CreatorWork) 생성	CREATOR	JSON, creator title(필수, 1~120자)

				description?(≤ 또는 /uploads/c externalUrl?(UI startedAt?/end endedAt≥starte
PATCH	/api/creator/works/{id}	본인 소유 CreatorWork 부분 수정	CREATOR + 소 유자 본인	JSON, creator 필드 optional(최 kind/descriptio 는 빈 문자열→nu startedAt/ende
DELETE	/api/creator/works/{id}	본인 소유 CreatorWork 삭제 (하드 딜리트)	CREATOR + 소 유자 본인	없음 (경로 파라미
GET	/api/creator/artworks	본인 판매 작품(Artwork) 목록 조회 (createdAt desc)	CREATOR	없음
POST	/api/creator/artworks	판매 작품(Artwork) 생성	CREATOR	JSON, artwork 수, 1~120), des imageUrl?(URL priceKrw(필수, ' ≥0, 기본 1), status(DRAFT/ 기본 DRAFT)
PATCH	/api/creator/artworks/{id}	본인 소유 Artwork 부분 수정	CREATOR + 소 유자 본인	JSON, artwork optional(최소 17 description/im priceKrw 양의 정 enum

DELETE	/api/creator/artworks/{id}	본인 소유 Artwork 삭제. 주문 (ArtworkOrder) 이력 있으면 하드 딜리트 대신 status=HIDDEN 소프트 처리	CREATOR + 소유자 본인	없음 (경로 파라미터 없음)
--------	----------------------------	---	------------------	-----------------

비고: uploads/route.ts 는 413(파일 크기 초과)을 반환하는 유일한 라우트다. 500은 어느 라우트에서도 명시적으로 반환하지 않으며, Prisma/fs 예외 시 Next.js 런타임이 암묵적 500을 발생시킨다. PUT 메서드를 export하는 라우트는 없다(수정은 모두 PATCH).

멤버십(Membership)

대상 라우트 4개 파일, HTTP 메서드 export 총 5개를 전수 문서화했습니다. 모든 행은 실제 코드 근거 기준입니다.

Method	Endpoint	기능	인증·역할	Request
POST	/api/membership-plans	멤버십 플랜 생성 (크리에이터 전용)	로그인필요 + CREATOR (getCurrentUser → role==="CREATOR" → creatorProfile 존재)	membershipPlanCreateSchema (validation/membership.ts): title(string 1~100, 필수), priceKrw(int 양수, 필수), description(string ≤500, 선택)
POST	/api/membership-plans/ai-suggest	멤버십 AI 가격·혜택 추천	로그인필요 + CREATOR (getCurrentUser → role==="CREATOR")	inline zod 스키마: description(string min1, 필수), category(string, 선택), targetAudience(string, 선택)
PATCH	/api/membership-plans/[id]	멤버십 플랜 수정	로그인필요 + CREATOR + 소유자 (role==="CREATOR" && creatorProfile 존재, 그리고	membershipPlanUpdateSchema (createSchema의 .partial()): title/priceKrw/description 모두 선택

		정 (본 인 소 유)	plan.creatorProfileId === 본인)	
DELETE	/api/membership-plans/[id]	멤 버 십 플 랜 삭 제 (본 인 소 유, 활 성 멤 버 없 을 때 만)	로그인필요 + CREATOR + 소유자 (role==="CREATOR" && creatorProfile 존 재, plan.creatorProfileId === 본인)	본문 없음 (_request 미사용), 경 로 파라미터 id
PATCH	/api/memberships/[id]/cancel	멤 버 십 구 독 취 소 (본 인 구 독)	로그인필요 (getCurrentUser null → 401, 역할 무 관). 소유권은 서비스 레 이어 cancelMembership에 서 검증	본문 없음 (_request 미사용), 경 로 파라미터 id(membershipId), user.id로 소유 검증

근거 메모 (정확성 확인용)

- **인증 패턴:** 모든 라우트가 @/lib/auth 의 getCurrentUser() 를 사용. requireUser / requireRole 헬퍼는 이 라우트들에서 사용되지 않고, 각 핸들러 내부에서 직접 null/role/creatorProfile 체크.
- **취소 라우트 역할 차이:** /api/memberships/[id]/cancel 은 CREATOR 역할 체크가 없음(라우트는 로그인만 확인). 실제 소유권 차단(403)·상태 차단(400)·존재 차단(404)·DB실패(500)은 cancelMembership 서비스가 반환하는 result.status 를 그대로 응답에 전달.
- **AI 추천 폴백:** 라우트의 catch가 실패 시에도 HTTP 200을 반환하도록 작성되어 있어(데모 무중단), 서비스 자체도 예외를 throw하지 않고 Mock으로 폴백. 따라서 이 엔드포인트의 구현상태는 Mock폴백.
- **활성 멤버 판정:** DELETE는 prisma.membership.count({ where: { planId: id } }) 로 서버 측 판정 (@MX:WARN 주석으로 명시). cancel 서비스의 isActiveMember 는 별도 함수이며 이 4개 라우트에서는 직접 호출되지 않음.

- **Membership 생성(가입/구독 시작) 라우트**는 대상 두 디렉터리에 존재하지 않음 — `membership-plans` 와 `memberships/{id}/cancel` 만 존재. 멤버 가입 엔드포인트는 이 범위 내 코드에 없으므로 지어내지 않음.

관련 파일 경로:

- `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/membership-plans/route.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/membership-plans/ai-suggest/route.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/membership-plans/{id}/route.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/app/api/memberships/{id}/cancel/route.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/validation/membership.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/membership.ts`
- `/Users/youngkwang/projects/artbridge/cadet-project/src/lib/ai/suggest.ts`

포스트·커뮤니티(Post/Community)

전수 대상 라우트 6개 / HTTP 메서드 export 7개. 모든 행은 실제 코드 근거.

Method	Endpoint	기능	인증·역할
POST	<code>/api/posts</code>	포스트 생성 (PUBLIC/MEMBER_ONLY/PAID)	로그인필요 + CREATOR(+creatorProfile 보유)
POST	<code>/api/posts/{id}/comments</code>	포스트 댓글 작성	로그인필요 + canViewPost 통과(작성자/PUBLIC/활성멤버/구매자)
POST	<code>/api/posts/{id}/likes</code>	포스트 좋아요 토글(없으면 추가, 있으면 취소)	로그인필요 + canViewPost 통과
POST	<code>/api/posts/{id}/purchase</code>	PAID 포스트 단건 구매 (Payment/Settlement/Notification 원자 생성)	로그인필요
POST	<code>/api/community-posts</code>	커뮤니티 글 생성	로그인필요 + canAccessCommunity 통과(활성멤버/결제완료 참여자/ 소유 크리에이터)
PATCH	<code>/api/community-posts/{id}</code>	커뮤니티 글 수정(부분 수정)	로그인필요 + canManagePost(작성자 본

			인 또는 스튜디오 소유 CREATOR)
DELETE	/api/community- posts/{id}	커뮤니티 글 삭제	로그인필요 + canManagePost(작성자 본 인 또는 스튜디오 소유 CREATOR)

비고:

- /api/posts 라우트에는 목록 조회용 GET export가 없음(POST만 존재). /api/community-posts 도 POST만, [id] 는 PATCH/DELETE만 존재(GET/PUT 없음). 전수 확인 결과 위 7개 메서드가 전부.
- 인증은 모두 getCurrentUser() 기반(null → 401). 별도 requireRole/requireUser 헬퍼는 미사용이며, 역할 체크는 라우트 내 인라인(user.role !== "CREATOR") 또는 access 헬퍼 (canViewPost/canAccessCommunity/canManagePost)로 수행.
- 구매 라우트의 Mock폴백은 의도된 안전 기본동작: purchasePost 가 환경변수 기반 resolvePaymentProvider() 가 아닌 고정 mockPaymentProvider 를 직접 호출하므로, PG 키 설정과 무관 하게 항상 Mock charge로 처리됨.

근거: src/app/api/{notifications,studio,auth}/**/*.route.ts 전수 Read + 서비스 (src/lib/notifications.ts , src/lib/queries/notifications.ts , src/lib/auth.ts , src/auth.ts , src/lib/prisma.ts) + 검증(src/lib/validation/studio.ts) 교차 확인. prisma 는 실 PrismaClient (Mock 아님)이며 세 도메인 모두 실제 DB 연동.

Notification (알림)

Method	Endpoint	기능	인증·역할	Request	Response
GET	/api/notifications	로그인 사용자의 알림 목록 (최신 순) 조회	로그인필요 (getCurrentUser() null→401)	본문 없음 (request 미사용)	200 Notification[] (createdAt desc)
PATCH	/api/notifications/{id}/read	개별 알림 읽음	로그인필요 (getCurrentUser() null→401)	path param id, 본문 없음	200 { id }

		표시 (본인 알림 만)			
PATCH	/api/notifications/read-all	사용자의 미읽음 알림 전체 읽음 처리	로그인필요 (getCurrentUser() null→401)	본문 없음	200 { count } (갯수 건수)

Studio (스튜디오)

Method	Endpoint	기능	인증·역할	Request	Response
PATCH	/api/studio	크리에이터 스튜디오 (CreatorProfile) 정보 편집	CREATOR (null→401, role≠CREATOR→403, 비소유자→403)	studioUpdateSchema(zod): crea studioName?(1~80), bio?(≤500 coverImageUrl?/profileImageU (url 또는 ""→null 해제)	

Auth (인증)

Method	Endpoint	기능	인증·역할	Request	Response
GET	/api/auth/[...nextauth]	Auth.js v5 인증 핸 들러(세션/CSRF/provider 콜백 등 catch-all)	공개 (NextAuth 내부 처리)	NextAuth 규약(엔드포인트별 상이)	NextAuth 표준 응답 (세션, JWT, 이력 등)
POST	/api/auth/[...nextauth]	Auth.js v5 인증 핸 들러(로그인/로그아웃/콜백 등 catch-all)	공개 (Credentials authorize: 이메일·비밀번호)	NextAuth 규약 (Credentials: email/password)	NextAuth 표준 응답 (세션, JWT 등)

			호 검증 후 발 급)		리다0 등)
--	--	--	----------------	--	-----------

참고(전수 검증 메모)

- 세 도메인의 실제 `route.ts` 는 총 4개 파일·6개 HTTP 메서드 export가 전부다(빠진 라우트/메서드 없음).
`studio · auth` 디렉터리에는 중첩 하위 라우트가 존재하지 않는다.
- `auth/[...nextauth]` 는 GET·POST 두 메서드만 export(`export const { GET, POST } = handlers`). PATCH/PUT/DELETE는 존재하지 않는다.
- 알림 라우트의 `request` 인자는 모두 미사용(`_request`)이라 본문 스키마가 없다.